

CAIKA-RAG-Typhoon

UIHxBNG Corporate AI Knowledge Assistant

Model Path ?

models/scb10x_llama3.2-typhoon2-t1-3

LLM Parameters ▼

Initialize CAIKA

Upload Documents

Upload corporate documents (PDFs)

Drag and drop files here

Limit 200MB per file • PDF

Browse files

UIHxBNG CAIKA-RAG-Typhoon

Your Corporate AI Knowledge Assistant

Ask a question about your corporate knowledge:

What is your model name?

Conversation

You: What is your model name?

CAIKA: My model name is "scb10x/llama3.2-typhoon2-t1-3b-research-preview-gguf".

<thoughts> <plan> <step order="1">Identifying the user's request</step> <step order="2">Locating the relevant information about the model name from the provided context</step> <step order="3">Formatting the response clearly and concisely while ensuring it is accurate based on the context provided.</step> </plan> <step> <title>Identifying the user's request</title> <scratch_pad> User wants to know the model name associated with their AI Knowledge Assistant. </scratch_pad> <summary>I'm focusing on the user's question regarding the model name for the AI Knowledge Assistant. </summary> <next_step>Locating the relevant information about the model name from the provided context</next_step> </step> <step> <title>Locating the relevant information about the model name from the provided context</title> <scratch_pad> In the provided context, it states: "My model name is 'scb10x/llama3.2-typhoon2-t1-3b-research-preview-gguf'." This clearly indicates that the user's AI Knowledge Assistant is indeed called "scb10x/llama3.2-typhoon2-t1-3b-research-preview-gguf". </scratch_pad> <summary>I've located the

Sources

Source 1: CAIKA-RAG-Typhoon Datasheet.pdf

Content preview ^

CAIKA-RAG-Typhoon Datasheet Overview

CAIKA-RAG-Typhoon (Corporate AI Knowledge Assistant) is a retrieval-augmented generation (RAG)

S...

Source 2: CAIKA-RAG-Typhoon Datasheet.pdf

Content preview ^

○ Chain Type: "stuff" (combines all relevant documents into a single context)

Technical Specifications

Performance Optimization

● ...

relevant information regarding the model name in the provided context.</summary>
 <next_step>Formatting the response clearly and concisely while ensuring it is accurate based on the context provided.</next_step> </step> <step> <title>Formatting the response clearly and concisely while ensuring it is accurate based on the context provided.</title>
 <scratch_pad> Final formatted response: "My model name is 'scb10x/llama3.2-typhoon2-t1-3b-research-preview-gguf'." </scratch_pad> <summary>I've crafted a clear and concise response to the user's question about their AI Knowledge Assistant's model name.
 </summary> <next_step></next_step> </step> </thoughts> <response> "My model name is 'scb10x/llama3.2-typhoon2-t1-3b-research-preview-gguf'." </response>
 #include "config.h"

```
#include <algorithm> #include <utility> #include <vector> #include <cmath>
```

```
#include "core/debugger/variable_value.h" #include "core/inspector/frontend/metadata/metadata_table.h" #include  

"core/inspector/frontend/metadata/metadata_column.h" #include  

"core/inspector/frontend/metadata/metadata_indexed_column.h" #include  

"core/inspector/frontend/metadata/metadata_array_column.h" #include  

"core/inspector/frontend/metadata/metadata_object_column.h"
```

```
using namespace Core::Inspector;
```

```
namespace { template <class T> struct ColumnInfo { const char *name; size_t size; // in bytes  

};
```

```
template <> struct ColumnInfo<char[]> { const char *name; size_t size; // in bytes };
```

```
const std::vector<ColumnInfo> &metadata_table() { static auto table =  

CreateMetadataTable(); return table; }
```

```
std::unique_ptr<MetadataTable> CreateMetadataTable() { std::vector<std::pair<ColumnId,  

ColumnInfo>>> columns;
```

```
// Adding metadata columns...  

columns.push_back(std::make_pair(ColumnId(0), 1), {"name", 256}));
```

Source 3: CAIKA-RAG-Typhoon Datasheet.pdf

Content preview



- fetch_k: Number to initially fetch before filtering
- score_threshold: Minimum similarity score
- Implementation Requirements
- System ...

```
columns.push_back(std::make_pair(ColumnId(1), 2), {"size", 10}));  
// Add more columns as needed  
  
// Create the MetadataTable  
auto table = std::make_unique<MetadataTable>(columns);  
  
return table;  
  
}  
  
} // namespace  
  
// Helper to create and populate a metadata table. std::unique_ptr<MetadataTable>  
CreateAndPopulateMetadataTable(const VariableValue &value) { const std::vector<char>  
&array_value = value.GetArray(); std::unique_ptr<MetadataTable> table(  
CreateMetadataTable());  
  
// Populate the array column with the provided data  
for (size_t i = 0; i < array_value.size(); ++i) {  
    table->AddRow(std::make_pair(array_value[i], sizeof(char)), {array_v  
}  
  
return table;  
  
} \section{Introduction}
```

The concept of a quantum computer is to use the principles of quantum mechanics to perform computations much more efficiently than classical
